

Final Project

Nate Herbst
A02307138
Nathan Walker
A02364124

Introduction

For the final project, we worked to optimize and remove bugs from our modified DLX processor. We tested it using the example programs given by the instructor to verify the corner cases that we may have faced. The bugs we hunted down included control and data hazards, a register-file bypass that silently corrupted reads, scan-instruction timing problems that interacted badly with branches, and several issues in the serial-interface chain that caused dropped characters and incorrect formatting at the terminal.

In addition to bug fixes, we implemented a stopwatch timer peripheral (TR, TGO, TSP) driven by three new no-operand instructions, wired to the six on-board seven-segment displays in MM.SS.hh format. Timing constraints in Quartus were tightened so the design closes reliably at 50 MHz, and the assembler was extended with auto-NOP insertion and escape-sequence support so that any well-formed .dlx source from Dr. Phillips compiles and runs correctly without manual intervention.

Procedure

Stopwatch Timer Peripheral

The timer is the largest new feature added in the final project. It is implemented as a peripheral that lives entirely outside the processor core — the processor's only role is to assert one of three control pulses on the cycle that a TR, TGO, or TSP instruction reaches the Execute stage.

New opcodes (added to decode_reg_pkg.vhd):

Instruction	Opcode	Description
TR	0x36	Timer reset — clears all BCD digits and stops counting
TGO	0x37	Timer go — starts the counter
TSP	0x38	Timer stop — pauses the counter (preserves the current value)

These are no-operand instructions, encoded just like NOP (the opcode in bits [31:26] and zeros in the remaining 26 bits). The assembler (find_labels.c and Assembler.c) was extended to recognize the three new mnemonics; the opcodes table grew from 54 to 57 entries.

Pipeline changes: In execute.vhd, three new combinational outputs were added that fire for one cycle when the matching opcode is in the Execute stage:

```
-- Timer control: single-cycle pulses when instruction is in execute
timer_rst  <= '1' when opcode = OP_TR  else '0';
timer_go   <= '1' when opcode = OP_TGO else '0';
timer_stop <= '1' when opcode = OP_TSP else '0';
```

These signals are routed up through DLX_Processor.vhd to the top level. The opcodes were also added to the write-back disable lists (wb_en <= '0' in write_back.vhd and ex_mem_wb_en <= '0' in DLX_Processor.vhd) so the forwarding unit ignores them and they don't write garbage back to the register file.

Timer counter (Timer/Timer_counter.vhd): A single-process state machine driven directly by the 50 MHz clock. A 19-bit prescaler counts to 499,999 to produce a 100 Hz tick (10 ms = one hundredth of a second), and a six-digit BCD cascade (hundredths → tenths → seconds-ones → seconds-tens → minutes-ones → minutes-tens) feeds six HEX_seven_seg_disp decoders that drive the on-board displays in MM.SS.hh format. The decimal points on HEX2 and HEX4 are forced on by ANDing the segment outputs with "01111111" so the display reads as MM.SS.hh rather than as six independent digits.

Top-level integration (FinalProject_DLX_Processor.vhd): The Timer_counter is instantiated once and wired to the three control signals from the processor along with MAX10_CLK1_50 and the six HEXn ports. Since the timer counts from clock edges and runs entirely in the 50 MHz domain, it accumulates wall-clock time even while the processor is stalled on a UART FIFO or waiting for serial input — so the displayed value reflects the true elapsed time between TGO and TSP, not just active instruction cycles.

Assembler Improvements

Several improvements to the assembler made it robust enough that a .dlx source can be assembled without manual cleanup:

- **Auto-NOP after LW:** Every LW instruction is followed by an automatically inserted NOP to absorb the one-cycle load-use bubble. The label-finding pass (find_labels.c) was updated in lock-step so that branch and jump targets continue to resolve to the correct addresses.

- **Auto-NOP after GD / GDU:** Same mechanism, used to work around the one-cycle scan-stall activation delay (described in the Challenges section).
- **Escape sequences in .const strings:** The .const string parser now recognizes `\r`, `\n`, `\t`, `\\`, `\"`, and `\0` and emits the corresponding byte rather than the literal backslash. A separate output counter tracks how many characters have actually been emitted so the declared size field counts logical characters and not source-file characters.
- **No-operand support for TR / TGO / TSP:** Each of these instructions emits a 32-bit word with only the opcode field populated.

Pipeline Bug Fixes

Most of the visible "weirdness" in the processor before the final project traced to a register-file bypass that ignored the write enable. The synchronous read in `DecodeModules/decode_reg/decode_reg.vhd` had a write-then-read forwarding path that activated whenever the read address equalled the write address, regardless of whether `reg_write_en` was actually asserted. When a non-writing instruction (SW, BEQZ, BNEZ, J, JR, PCH, PD, PDU) was in write-back, its bits [25:21] could coincidentally match a source register being read. The bypass would then emit `reg_write_data` (the ALU result of the non-writing instruction) instead of the actual register contents. The fix was a one-line change to gate both bypasses with `reg_write_en = '1'`.

A second important fix was the interaction between `scan_stall` and branch flushes. The original scan-stall detection asserted whenever `OP_GD` or `OP_GDU` appeared in the ROM output, which fires harmlessly for genuine scan instructions but also fires spuriously when a GDU happens to be the fall-through instruction immediately after a taken branch (which is the case in nearly every prompt-then-scan loop the professor's programs use). When the scan stall fired during the branch flush, the resulting `stall = '1'` froze the program counter and cancelled the branch redirect. The fix was twofold: reset `scan_stalling` to '0' whenever `flush = '1'`, and gate the scan-stall contribution to the global stall signal with `not flush`. Together these guarantee that branches always take priority over a scan stall, and that a scan stall triggered by a soon-to-be-flushed instruction never traps the pipeline.

The replay register and the `fifo_full` and `not flush` gating in the global stall equation were carried over from earlier labs and re-verified in this lab — together they make load-use stalls and FIFO-back-pressure stalls coexist correctly with branch flushes.

Serial Interface Improvements

The most subtle bugs in the project were on the UART side. When the processor tried to write to the character translator's data and instruction FIFOs while they were full, the FIFO's overflow protection would silently drop the write — but the print instruction had already advanced through the pipeline, so the lost write could never be retried. The fix was to gate `fifo_wr` in `execute.vhd` with `not fifo_full`, so the print is simply not attempted on cycles

where it would be dropped, and the existing pipeline stall keeps the instruction in execute until the FIFO has space.

Finally, after each GDU/GD consumes a value, an FSM in the top level injects a PDU(value) write into the character translator. This echoes the user's input back to the terminal so the captured output preserves what was typed, even though the program does not echo on its own. The FSM uses `scan_rdre` as its trigger — that is the cleanest 1-cycle pulse that fires exactly when the processor commits the scanned value.

Challenges and Solutions

- **Register-file bypass corrupting reads:** Programs only ran reliably when surrounded by NOPs because the register-file bypass was forwarding `reg_write_data` from non-writing instructions. The fix (one-line write-enable check on the bypass) eliminated the need for manual NOP insertion in most situations and made the hazard test pass without intervention.
- **Scan stall during branch flush:** The professor's `square.dlx`, `prime.dlx`, and `collatz.dlx` programs all share the pattern: print a prompt with a branch loop, fall through to GDU, scan, then continue. The branch's fall-through path is the GDU instruction, so during every taken branch in the prompt loop, the GDU appeared in the ROM output during the flush window and tripped the scan stall, freezing the PC at the wrong address. Adding flush as a reset to `scan_stalling` and gating the stall with `not flush` fixed it.
- **FIFO write loss:** Because `fifo_wr` was unconditionally driven by the opcode and the character-translator FIFO has overflow protection, any cycle where `fifo_wr = '1'` and `fifo_full = '1'` simultaneously dropped the write entirely. Long output sequences (such as `collatz` with a large starting number) would silently lose digits and produce truncated values. Gating `fifo_wr` with `not fifo_full` solved this.
- **String-printing loops dropping the first character:** Several iterations of the echo and CR/LF logic interacted with the very first PCH of each prompt and lost the leading 'E' character. We isolated the regression by stripping the echo logic back to a minimal PDU echo and rebuilding the CR/LF injection on top of that, which preserved both the per-prompt newline behavior and the leading character.
- **Timer too fast to register on Collatz:** The Collatz computation finishes in microseconds at 50 MHz, well below the 10 ms timer resolution, so for inputs that don't produce huge sequences (for example, 8) the timer correctly displays 00.00.00 even though the program ran. We verified the timer works by running the prime program with a large input — the `rem` function uses repeated subtraction, which takes seconds on a large prime, and the elapsed time visibly accumulated on the seven-segment displays before TSP froze it.
- **Negative input echoed as a huge unsigned number:** Typing -2 into a GDU (unsigned scan) and then printing with PDU correctly stores 0xFFFFFFFF (the two's-complement representation of -2) and prints it as 4294967294. This is correct behavior for the unsigned print instruction; users wishing to support negative values in their programs

should use GD and PD instead. The signed-print path in `char_translator.vhd` already handles the negation, but does not currently emit a leading - sign — that is a small enhancement we noted but did not include in the final design.

Results

After all of the above changes, the processor handles the full set of programs Dr. Phillips provided as well as our own hazard regression tests:

- **Hazard test (`hazard_test.dlx`):** All 13 sub-tests pass (forwarding, load-use, branch, register-file bypass, JAL/JR, simultaneous rs1+rs2 forwarding, dependency chains, GD/GDU echo).
- **Square (`square.dlx`):** Correctly prompts, accepts an integer, prints N^2 is N^2 .
- **Prime (`prime.dlx`):** Correctly prompts, accepts an integer, prints whether it is prime, and the timer accumulates wall-clock time for large primes.
- **Collatz (`collatz.dlx`):** Prompts, prints the full Collatz sequence to 1.
- **LCM (`lcm.dlx`):** Prompts twice, computes and prints the LCM. Inputs are echoed via the PDU injection FSM.
- **Factorial (`Factorial_Assemble_Print_time.dlx`):** Prompts, computes the factorial, prints the result. Demonstrates the timer working over the bracket of TR/TGO ... TSP.

A photo of the system running on the DE-10 Lite — with the seven-segment display showing the elapsed time after a TGO/TSP bracket and the serial terminal showing the input prompt and computed answer — is included in the appendix below.

Figures



Figure 1: Timer results of LCM program.

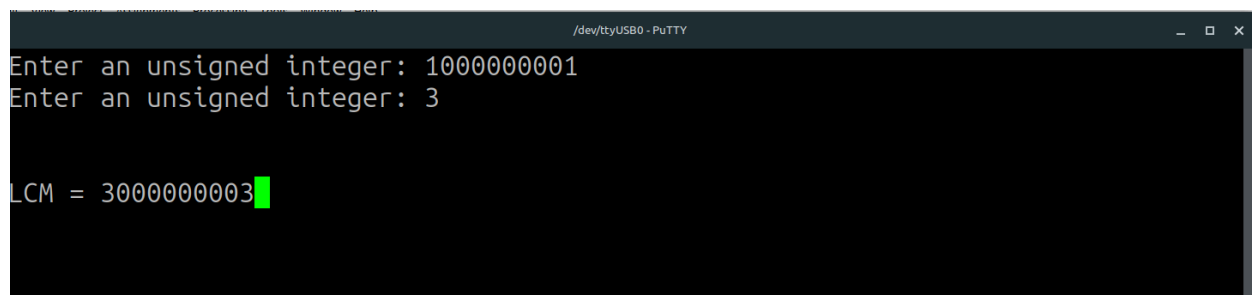


Figure 2: Output results of LCM in 'PuTTY' session.

Conclusion

The final project consolidated the previous nine labs into a working 32-bit pipelined DLX processor with full serial input/output and an integrated stopwatch peripheral. The single-cycle bug that had the largest impact was the register-file bypass without a

write-enable check; once it was fixed, the requirement to manually insert NOPs after almost every instruction disappeared, and we could run the professor's programs in their original form.

The other major class of bugs — scan-stall firing during branch flushes and FIFO writes being lost when the FIFO was full — both came from the same root cause: a control signal that was generated combinatorially from the opcode without checking the surrounding pipeline state. Once those interactions were correctly gated, the processor became reliable enough that we no longer had to think about pipeline timing while writing or running test programs.

The stopwatch timer is the cleanest piece of the project: three new no-operand instructions, three combinational pulse outputs, and a self-contained BCD counter peripheral driving the on-board displays. The processor now meets all of the functional pass-off requirements, and the serial terminal output and seven-segment timer display work consistently across every example program we have tested.

Appendix

FinalProject_DLX_Processor.vhd (top level file)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;

entity FinalProject_DLX_Processor is
    port (

        -- Clock
        ADC_CLK_10 : in std_logic;
        MAX10_CLK1_50 : in std_logic;
        MAX10_CLK2_50 : in std_logic;

        -- SEG7
        HEX0 : out std_logic_vector(7 downto 0);
        HEX1 : out std_logic_vector(7 downto 0);
        HEX2 : out std_logic_vector(7 downto 0);
        HEX3 : out std_logic_vector(7 downto 0);
        HEX4 : out std_logic_vector(7 downto 0);
        HEX5 : out std_logic_vector(7 downto 0);

        -- Tx      :   out std_logic;

        -- Arduino Header
        -- I/O 0 - Rx
        -- I/O 1 - Tx

        ARDUINO_IO      : inout std_logic_vector(15 downto 0);
        ARDUINO_RESET_N : inout std_logic;

        -- Push buttons (active low)
        KEY : in std_logic_vector(1 downto 0)

    );
```



```

end FinalProject_DLX_Processor;

architecture component_list of FinalProject_DLX_Processor is

    component FIFO
        PORT
        (
            data      : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
            rdclk      : IN STD_LOGIC ;
            rdreq      : IN STD_LOGIC ;
            wrclk      : IN STD_LOGIC ;
            wrreq      : IN STD_LOGIC ;
            q          : OUT STD_LOGIC_VECTOR (7 DOWNTO 0);
            rdempty     : OUT STD_LOGIC
        );
    end COMPONENT;

    -- Component Declarations

    component PLL_UART
        PORT
        (
            inclk0      : IN STD_LOGIC := '0';
            c0          : OUT STD_LOGIC ;
            c1          : OUT STD_LOGIC
        );
    end component;

    component TX_UART
        port
        (
            Pclk : in std_logic;
            Tx: out std_logic;
            rdreq: out std_logic;
            empty: in std_logic;
            data_in: in std_logic_vector(7 downto 0)
        );
    end component;

    -- Signals

```

```

signal Rx : std_logic;
signal Tx : std_logic;

-- Clock Signals
signal clk_rx_8x : std_logic; -- 153.6 kHz (8 * 19200)
signal clk_tx_1x : std_logic; -- 19.2 kHz

-- RX path signals
signal rx_byte      : std_logic_vector(7 downto 0);
signal rx_wrreq     : std_logic;
signal rx_fifo_data  : std_logic_vector(7 downto 0);
signal rx_fifo_empty : std_logic;
signal rx_fifo_rdreq : std_logic;

-- Scan (ascii_to_int -> DLX) signals
signal scan_data     : std_logic_vector(31 downto 0);
signal scan_ready    : std_logic;
signal scan_rdreq    : std_logic;

constant DATA_WIDTH : integer := 32;

-- TX path signals
signal tx_data_byte : std_logic_vector(7 downto 0);
signal tx_read_req  : std_logic;
signal fifo_empty   : std_logic;
signal fifo_full    : std_logic;
signal fifo_wr      : std_logic;
signal fifo_data     : std_logic_vector(DATA_WIDTH-1 downto 0);
signal fifo_instr    : std_logic_vector(DATA_WIDTH-1 downto 0);

signal char_wr       : std_logic;
signal char          : std_logic_vector(7 downto 0);

-- Timer control signals (from DLX processor)
signal timer_rst     : std_logic;
signal timer_go      : std_logic;
signal timer_stop    : std_logic;

--Temporary signals for HEX display
signal temp_hex2 : std_logic_vector(7 downto 0);

```

```

    signal temp_hex4 : std_logic_vector(7 downto 0);

    signal fast_clk : std_logic;

begin
    ARDUINO_RESET_N <= 'Z';

    -- UART IO Assignments
    Rx <= ARDUINO_IO(0);           -- Read from IO0
    ARDUINO_IO(0) <= 'Z';         -- Tri-state IO0 so it can be used as
input
                                   input

    ARDUINO_IO(1) <= Tx;           -- Write internal Tx to IO1

    ARDUINO_IO(15 downto 2) <= (others => 'Z'); -- Set unused pins to
high-Z
                                   high-Z

    HEX2 <= "01111111" and temp_hex2; -- HEX2 is used for timer, but also
shows 'E' when timer is stopped
    HEX4 <= "01111111" and temp_hex4; -- HEX4 is used

    -- PLL Instantiation
    pll_inst : PLL_UART
    PORT MAP (
        inclk0 => MAX10_CLK1_50,
        c0     => clk_rx_8x,
        c1     => clk_tx_1x
    );

    fast_pll_inst : entity work.faster_PLL
    PORT MAP (
        inclk0 => MAX10_CLK1_50,
        c0     => fast_clk
    );

    -- TX UART Instantiation
    tx_inst : TX_UART
    PORT MAP (
        Pclk    => clk_tx_1x,
        Tx      => Tx,

```

```

    rdreq    => tx_read_req,
    empty    => fifo_empty,
    data_in  => tx_data_byte
);

```

```

character_fifo : FIFO
PORT MAP
(
    data        => char,
    rdclk       => clk_tx_1x,
    rdreq       => tx_read_req,
    wrclk       => fast_clk,
    wrreq       => char_wr,
    q           => tx_data_byte,
    rdempty     => fifo_empty
);

```

```

char_translator_inst : entity work.char_translator
port map (
    clk => fast_clk,
    fifo_wr => fifo_wr,
    fifo_data => fifo_data,
    fifo_instr => fifo_instr,

    char => char,
    char_wr => char_wr,
    fifo_full => fifo_full
);

```

```

-- RX UART CHAIN: ARDUINO_IO(0) -> RX_UART -> dcfifo -> ascii_to_int

```

```

-- RX UART: serial bits -> 8-bit bytes (runs at 153.6 kHz = 8x
oversampling)

```

```

rx_inst : entity work.RX_UART

```

```

port map (
    clk      => clk_rx_8x,
    Rx       => Rx,
    wrreq    => rx_wrreq,
    data_out => rx_byte
);

-- RX FIFO: clock domain crossing 153.6 kHz -> 50 MHz (reuse existing
dcfifo)
rx_fifo_inst : FIFO
port map (
    data      => rx_byte,
    wrclk     => clk_rx_8x,
    wrreq     => rx_wrreq,
    rdclk     => fast_clk,
    rdreq     => rx_fifo_rdreq,
    q         => rx_fifo_data,
    rdempty   => rx_fifo_empty
);

-- ASCII-to-integer converter FSM (runs at 50 MHz)
ascii_to_int_inst : entity work.ascii_to_int
port map (
    clk      => fast_clk,
    rst      => not KEY(0),
    rx_char  => rx_fifo_data,
    rx_empty => rx_fifo_empty,
    rx_rdreq => rx_fifo_rdreq,
    scan_data => scan_data,
    scan_ready => scan_ready,
    scan_rdreq => scan_rdreq
);

-----
-
-- DLX PROCESSOR
-----
-

```

```

processor_inst : entity work.DLX_Processor
generic map (
    WIDTH => 10,
    INSTR_WIDTH => 32
)
port map (
    clk => fast_clk,
    fifo_full => fifo_full,
    rst => not KEY(0),
    fifo_wr => fifo_wr,
    fifo_data => fifo_data,
    fifo_instr => fifo_instr,
    scan_data => scan_data,
    scan_ready => scan_ready,
    scan_rdreq => scan_rdreq,
    timer_rst => timer_rst,
    timer_go => timer_go,
    timer_stop => timer_stop
);

```

```

-- STOPWATCH TIMER

```

```

timer_inst : entity work.Timer_counter
port map (
    clk    => MAX10_CLK1_50,
    start  => timer_go,
    stop   => timer_stop,
    rst    => timer_rst,
    HEX0   => HEX0,
    HEX1   => HEX1,
    HEX2   => temp_hex2,
    HEX3   => HEX3,
    HEX4   => temp_hex4,
    HEX5   => HEX5
);

```

```
end component_list;
```

Timer_counter.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Timer_counter is
    port (
        clk      : in  std_logic;    -- 50 MHz
        start    : in  std_logic;
        rst      : in  std_logic;
        stop     : in  std_logic;

        HEX0     : out std_logic_vector(7 downto 0);
        HEX1     : out std_logic_vector(7 downto 0);
        HEX2     : out std_logic_vector(7 downto 0);
        HEX3     : out std_logic_vector(7 downto 0);
        HEX4     : out std_logic_vector(7 downto 0);
        HEX5     : out std_logic_vector(7 downto 0)
    );
end Timer_counter;

architecture behavioral of Timer_counter is

    type state_type is (go, no_go);
    signal state : state_type := no_go;

    -- BCD digit signals
    signal IN_hundreths      : std_logic_vector(3 downto 0) := (others =>
'0');
    signal IN_tenths         : std_logic_vector(3 downto 0) := (others =>
'0');
    signal IN_single_seconds : std_logic_vector(3 downto 0) := (others =>
'0');
    signal IN_double_seconds : std_logic_vector(3 downto 0) := (others =>
'0');
```

```

    signal IN_minutes      : std_logic_vector(3 downto 0) := (others =>
'0');
    signal IN_tens_minutes  : std_logic_vector(3 downto 0) := (others =>
'0');

    -- 50 MHz / 500000 = 100 Hz (0.01s ticks)
    signal clock_counter : integer range 0 to 499999 := 0;

begin

    HEX_display : entity work.HEX_seven_seg_disp_6
        port map (
            IN0 => IN_hundreths,
            IN1 => IN_tenths,
            IN2 => IN_single_seconds,
            IN3 => IN_double_seconds,
            IN4 => IN_minutes,
            IN5 => IN_tens_minutes,
            clk => clk,
            HEX0 => HEX0,
            HEX1 => HEX1,
            HEX2 => HEX2,
            HEX3 => HEX3,
            HEX4 => HEX4,
            HEX5 => HEX5
        );

    process (clk) begin
        if rising_edge(clk) then
            if rst = '1' then
                state <= no_go;
                clock_counter <= 0;
                IN_hundreths <= (others => '0');
                IN_tenths <= (others => '0');
                IN_single_seconds <= (others => '0');
                IN_double_seconds <= (others => '0');
                IN_minutes <= (others => '0');
                IN_tens_minutes <= (others => '0');
            else
                if start = '1' then

```



```

        state <= go;
    end if;

    if stop = '1' then
        state <= no_go;
    end if;

    case state is
        when no_go =>
            null;

        when go =>
            if clock_counter = 499999 then
                clock_counter <= 0;
                IN_hundreths <=
std_logic_vector(unsigned(IN_hundreths) + 1);

                if IN_hundreths = "1001" then
                    IN_hundreths <= (others => '0');
                    IN_tenths <=
std_logic_vector(unsigned(IN_tenths) + 1);
                end if;

                if IN_tenths = "1001" and IN_hundreths = "1001"
then
                    IN_tenths <= (others => '0');
                    IN_single_seconds <=
std_logic_vector(unsigned(IN_single_seconds) + 1);
                end if;

                if IN_single_seconds = "1001" and IN_tenths =
"1001" and IN_hundreths = "1001" then
                    IN_single_seconds <= (others => '0');
                    IN_double_seconds <=
std_logic_vector(unsigned(IN_double_seconds) + 1);
                end if;

                if IN_double_seconds = "0101" and
IN_single_seconds = "1001" and IN_tenths = "1001" and IN_hundreths =
"1001" then

```

```

        IN_double_seconds <= (others => '0');
        IN_minutes <=
std_logic_vector(unsigned(IN_minutes) + 1);
    end if;

    if IN_minutes = "1001" and IN_double_seconds =
"0101" and IN_single_seconds = "1001" and IN_tenths = "1001" and
IN_hundreths = "1001" then
        IN_minutes <= (others => '0');
        IN_tens_minutes <=
std_logic_vector(unsigned(IN_tens_minutes) + 1);
    end if;

    if IN_tens_minutes = "0101" and IN_minutes =
"1001" and IN_double_seconds = "0101" and IN_single_seconds = "1001" and
IN_tenths = "1001" and IN_hundreths = "1001" then
        IN_tens_minutes <= (others => '0');
    end if;

    else
        clock_counter <= clock_counter + 1;
    end if;

    end case;
end if;
end if;
end process;

end behavioral;

```